



Enhancing Explainability in Credit Scoring with Interpretable Machine Learning

XAI-FIN-2023 Submission 6



Disclaimer: All errors are mine.

Enhancing Explainability in Credit Scoring with Interpretable Machine Learning

DENIS BURAKOV

In this paper, we show, using a synthetic dataset created by blending real-world credit risk data, that when responsibly designed and properly constrained, Machine Learning (ML) models used for credit scoring – under certain conditions – outperform their linear alternatives both in terms of discriminatory power and calibration accuracy while remaining highly interpretable. The design of ML models discussed in this paper may provide a compelling case for conservative lenders to begin transitioning toward more advanced algorithms without having to compromise on the balance between explainability and accuracy.

1 INTRODUCTION

Credit scoring is a regulated, high-risk application of machine learning used primarily in Financial Services. Credit scoring became one of the most successful predictive modeling use-cases and revolutionized consumer lending helping banks to immediately become more profitable [3]. Areas of application of scoring models include loan application and behavioral scoring, risk segmentation, and pricing among other internal uses.

Notwithstanding economic gains, credit scoring models are replete with model risk. An incorrect use of a credit scoring model (e.g., caused by poor model design or low quality and non-representative data) can harm customers at scale and expose a lender to legal risks. As a result, comprehensive regulatory frameworks have been established to safeguard consumers from the failure of automated decision-making.

In banking, the golden standard of a standardized credit risk assessment is the logistic regression model, often advanced via a supervised binning technique called Weight-of-Evidence (WOE). A "gold standard" scorecard produced via this modeling technique is an additive model with a small number of main effects (deviations from this may result in less interpretable models). Therefore, such models may struggle to capture all the complex interactions within the data, which results in costly manual feature engineering to improve model performance.

In other domains, researchers frequently emphasize the superior performance of tree-based models, particularly gradient boosted trees, over linear [4] and deep learning models on tabular data [13]. Despite the impressive success in other domains, the adoption of tree-based models for risk management purposes lags behind their scope of application in other fields due to reasons of data quality, explainability, and IT-infrastructure [21].

The key reason behind this gap, primarily for credit risk management [21] and in the Financial services in general [11], is a prevailing sentiment among regulators and model risk managers that Boosting / Random Forest models are intrinsically opaque or "black-box" thus rendering them unsuitable for high-stakes decision-making like lending. This concern has led to various efforts to enhance the interpretability of "black-box" models using tools like SHAP¹ or surrogate modeling. In response, some ML practitioners have expressed concerns about the overall use of "black-box" ML for high-risk applications, as advocated by C. Rudin [22].

In this paper, building on prior studies [9], [8], [15], we show that tree-based credit scoring models can offer interpretability without compromising performance, challenging prevailing industry opinions. Utilizing a common scorecard methodology for explaining models to frontline staff, auditors, and regulators, we assess two tree-based model variants: Constrained² XGBoost and an extension of XGBoost with random forests as base learners. These models are

¹SHapley Additive exPlanations: <https://shap.readthedocs.io/en/latest/>

²We adopt the term constrained from [15] to describe tree-based models that can incorporate constraints (e.g., monotonic and/or interaction constraints).

benchmarked against a classic WOE Logistic Regression (WOE LR) model, using performance metrics like the Gini score and an extension of Expected Calibration Error (ECE) [14].

Our goal is to illustrate that well-configured and effectively communicated tree-based ensemble models are suitable for credit scoring and can exhibit similar characteristics to linear scoring models. The methodology of credit risk modeling presented in this paper aims to shift the perception of tree-based ensemble models from being considered unsuitable to becoming an industry standard for assessing consumer credit risk.

2 DATASET ENGINEERING

In this paper, a synthetic credit risk portfolio is engineered by combining data from three primary sources:

dataset_A: FICO Explainable ML Challenge³.

- Origin: FICO Explainable ML Challenge.
- Loan purpose: Line-of-credit loans.
- Target Variable: 90 days past due (DPD) over 24 months.

dataset_B: Lending Club⁴.

- Source: Lending Club.
- Loan purpose: Line-of-credit and installment loans.
- Target Variable: 90 days past due (DPD) over 24 months.

dataset_C: Give Me Some Credit⁵.

- Source: Kaggle.
- Loan purpose: Line-of-credit and installment loans.
- Target Variable: 90 days past due (DPD) over 24 months.

The first dataset describes the application scoring process (applying for a home credit line), the other two describe behavioral scoring (customers who were approved to use a lending product) and include line-of-credit and installment product types. Combined together, they form a synthetic end-to-end credit scoring dataset incorporating application and behavioral stages of the loan lifecycle. In addition, a similar operationalization of the target variable in the three datasets allows to model the horizon of into-default event consistently.

Risk drivers include balances and credit line utilization, credit bureau scores, socio-demographic information from the application form as well as some other indicators related to a product mix of a customer. A full list of selected risk drivers is available in Table 4, including monotonic constraints (business logic) and inclusion of specific features into the blended dataset. We use 10% default rate to reduce the effect of class imbalance on predictive accuracy following recent research in this area [7].

An experimental dataset generation process consists of two major steps: random sampling of defaulted labels from the three datasets and stacking them together with randomly sampled features across horizontal axis as shown in Figure 1. This approach can be extended to generate datasets with other target variables relevant for credit risk modeling (e.g., loss given default, credit loss) using stratification based on a binary target variable.

Below, we outline a dataset generation process for defaulted records from the three scoring datasets:

³FICO-xML-Challenge Data: <https://community.fico.com/s/explainable-machine-learning-challenge>, <https://github.com/Trusted-AI/AIX360/blob/master/examples/tutorials/HELOC.ipynb>, https://pbiecek.github.io/xai_stories/story-heloc-credits.html

⁴Lending Club Data: https://www.openintro.org/data/index.php?data=loans_full_schema, <https://www.sec.gov/Archives/edgar/data/1409970/000089161808000318/f41480orsv1.htm>

⁵Give Me Some Credit Data: <https://www.kaggle.com/competitions/GiveMeSomeCredit/>

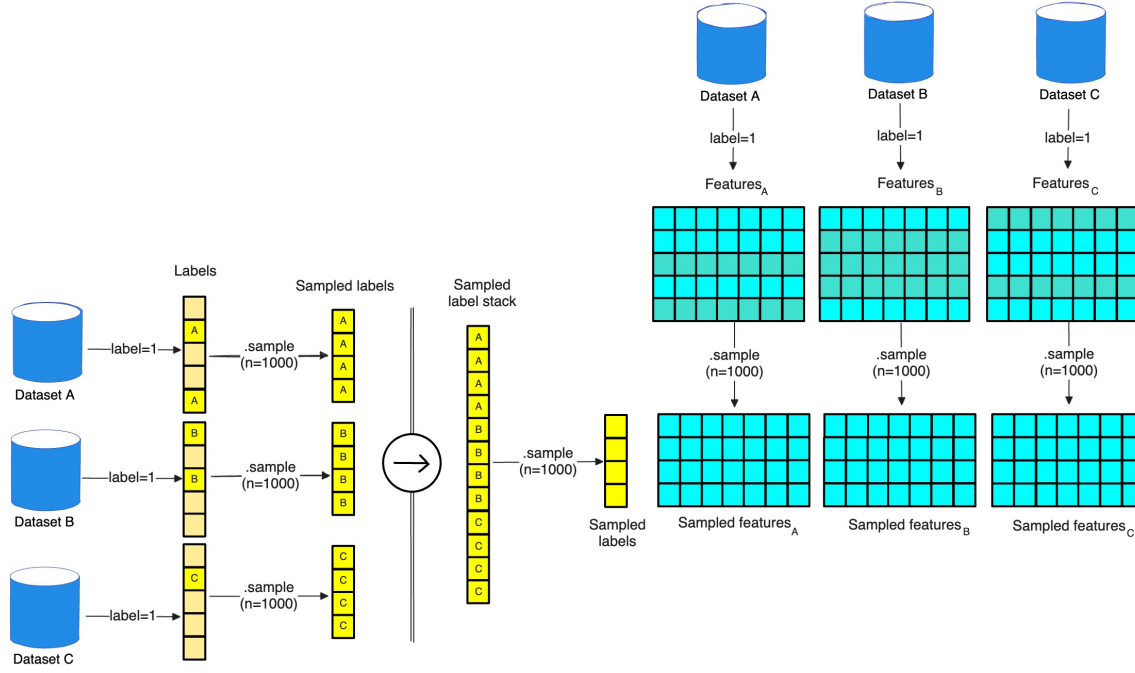


Fig. 1. Dataset engineering process (for defaulted records).

The resulting dataset is composed of randomly sampled labels and features from the three datasets. This particular approach allows to include more columns with risk drivers that describe defaulted (or non-defaulted) customers' behavior and are no longer dataset-specific after a blending procedure.

Similarly, we generate a second dataset for non-defaulted records following the same logic and a sample size of 9000. Combining the defaulted and non-defaulted datasets together allows us to create a synthetic scoring dataset with a 10% default rate to be used in the modeling process.

3 MODELS

3.1 Baseline: WOE Logistic Regression

The golden standard of a standardized risk assessment is a Weight-of-Evidence Logistic Regression (WOE LR) model. This model design has been in use for several decades and continues to be among the most interpretable and reliable methods in credit risk modeling. The outputs of WOE LR can be easily converted into a scorecard format helping to facilitate explainability of such models to end users.

Supervised binning is the key component of a WOE LR model. During this process, a model developer transforms the raw values of candidate features into several bins, according to some business logic or splitting techniques (e.g., decision

trees). The main idea of supervised binning is to find optimal cut-off points to define bins that ensure a balance between being "wide enough" and "narrow enough" by having distinctly different risk characteristics with minimum information loss [16]. Feature transformation consists in mapping its raw values to a bin's corresponding Weight-of-Evidence (WOE) score.

For each feature, WOE score for bin b_i is calculated as follows:

$$WOE_{b_i} = \log \left(\frac{\%good_{b_i}}{\%bad_{b_i}} \right) \quad (1)$$

In Equation 1, the numerator corresponds to the share (%) of good (non-defaulted) records in a bucket b_i out of the total number of good (non-defaulted) records, and the denominator corresponds to the share (%) of bad (defaulted) observations in a bucket b_i out of the total number of bad (defaulted) observations in the sample.

The Weight-of-Evidence (WOE) score can take on any value. Positive WOE scores indicate that a particular bin has a lower default rate compared to the sample average default rate and has a higher default rate if WOE scores are negative. In machine learning terminology, a vector of WOE scores as an encoded feature can be thought of as a "weak learner", akin to a univariate decision tree [9].

After supervised binning is performed and features are transformed into WOE scores, a logistic regression model is estimated. The objective function of the model is to minimize the logistic cost function, or log loss (Equation 2), using Maximum Likelihood Estimation (MLE). The output of the logit function (Equation 3) is subsequently transformed into probabilities using a sigmoid (Equation 4) function.

$$\text{Log Loss} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)] \quad (2)$$

Log Loss (Equation 2) measures the accuracy of a credit scoring model by comparing its predicted probabilities to the actual outcomes. N represents the total number of data points, i is an index representing each data point, y_i is the actual binary outcome (0 or 1) for the i -th data point, and p_i is the predicted probability that the i -th data point belongs to the positive class (class 1). The Log Loss penalizes the model when it predicts the wrong class with a high probability.

$$\text{Logit} (P(Y = 1)) = \sum_{j=1}^M \beta_0 + \beta_j W_j(x_j) \quad (3)$$

In Equation 3, $P(Y = 1)$ represents the probability that the default event Y is equal to one. M is the total number of features, j is an index representing each feature, β_0 is the intercept term in logistic regression, and β_j represents a coefficient for feature j . This equation calculates the log-odds (logit) of the probability $P(Y = 1)$ as a linear combination of risk drivers.

$$P(Y = 1) = \frac{1}{1 + e^{-\sum_{j=1}^M \beta_0 + \beta_j W_j(x_j)}} \quad (4)$$

Equation 4 uses the sigmoid function to map a linear combination of features (logit) to a probability value between 0 and 1.

In accordance with Equation 1, all coefficients in the logistic regression model are designed to be negative (negative WOE scores multiplied by negative weights increase default probability)⁶. Features that flipped signs may be signs of

⁶Unless $P(Y = 0)$ is modeled, in this case all coefficients must be positive.

overfitting and can be treated with Lasso (L1) feature selection or via dropping such features from the model, among other methods.

We estimate a WOE LR using OptBinning⁷ package that uses a specific set of constraints to be optimized during the supervised binning procedure. These constraints might range from requiring a minimum number of records per bin to monotonicity constraints. This variant of the binning process is known as the optimal binning process [19]. The optimal binning is generally solved by iteratively merging an initial granular discretization until imposed constraints are satisfied.

Default parameters were used in the binning procedure starting with the maximum number of bins after pre-binning (`max_n_prebins`) being 20 and the fraction of minimum number of records for each prebin (`min_prebin_size`) being 5%. Additionally, expert-based monotonicity constraints were incorporated into the binning process via the `binning_fit_params` parameter.

3.2 Model 1: Constrained XGBoost

XGBoost is an advanced Gradient Tree Boosting-based software that can efficiently handle large-scale ML tasks which has become the de facto method-of-the-first-choice for a majority of data science problems (Wang et al. 2020). XGBoost is made up of several decision trees, each of which is a weak learner, and Boosting technology promotes it to a strong learner [25].

In the context of binary classification, unless specified differently, XGBoost uses log loss (Equation 2) as an objective function. Binary classification trees are constructed sequentially, forming an ensemble of weak learner trees with the objective to minimize loss. Each successive tree in the ensemble aims to correct errors made by the previous trees by working on the first (gradients) and second order (hessians) gradient statistics of the previous tree's predictions.

Each individual tree in the binary classification case produces log-odds scores (margins) for each tree's leaf nodes, which are subsequently added together to produce the final prediction (Equation 5). A sigmoid transformation (Equation 4) is used to convert the resulting sum of margins into probabilities.

An XGBoost tree ensemble model can be viewed as a function of the following form:

$$G(P(Y = 1)) = \sum_{j=1}^M H_j(x_j) \quad (5)$$

Where $H_j(x_j)$ represents an individual tree j that is combined into the final prediction ($G(P(Y = 1))$).

XGBoost's objective function optimized during training can be expressed using Equation 6 adapted from [6]:

$$\text{obj}(t) = \sum_{j=1}^M \left(y_i, \hat{y}_i^{(t-1)} + H_j(x_j) \right) + \Omega(H_j(x_j)) \quad (6)$$

Where Ω is a regularization term to control for complexity of trees to avoid overfitting.

The newer version of XGBoost, referred to as Constrained XGBoost, incorporates monotonic constraints [20], allowing users to enforce monotonic relationships between model inputs and predictions [15]. These constraints prevent splits on monotonic features that could violate the constraints, enabling the incorporation of causal knowledge (e.g., monotonic relationships) by domain experts into the modeled relationships (e.g., higher debt-to-income ratios indicating higher risk).

⁷OptBinning: The Python Optimal Binning library <http://gnpalencia.org/optbinning/>

Another customization option for XGBoost models involves using random forest base learners. These learners are particularly appealing for credit risk modeling as they enable the creation of multiple sub-models at each tree iteration in gradient boosting. Domain experts can assess various randomly constructed trees to derive new features based on variable interactions within trees or aggregate predictions from promising tree candidates into a new model.

Random forests may be implemented in XGBoost as base learners by setting `num_parallel_tree` parameter equal to an integer greater than 1. The idea is that each boosting round will no longer consist of one tree, but a number of parallel trees, which in turn make up a forest [24]. It is important to note in this case that the number of trees being constructed is derived as `num_parallel_tree` $\times$ `n_estimators`. Therefore, with 50 iterations and 2 trees per iteration we will have built 100 trees.

Table 1. Constant Hyperparameter Values

Hyperparameter	Value
<code>base_score</code>	0.1
<code>eval_metric</code>	auc, logloss
<code>early_stopping_rounds</code>	10
<code>monotone_constraints</code>	User-defined list
<code>tree_method</code>	hist
<code>missing</code>	np.nan
<code>random_state</code>	24

The following hyperparameters were optimized for the two tree-based model candidates using stratified 5-fold cross-validation using Optuna⁸ framework. We use a combination of L1 (controlled by `alpha` parameter) and L2 (controlled by `lambda` parameter) penalization of weights to reduce complexity. The number of iterations for Constrained RF XGBoost was reduced to 30 with a maximum of `num_parallel_tree` of 3 this would result in 90 trees as a maximum number of trees to be estimated, which is similar to Constrained XGBoost (100). Additionally, `num_parallel_tree` was kept low between 2 and 3 for Constrained RF XGBoost in order to avoid overfitting.

Table 2. Hyperparameter Ranges

Hyperparameter	Constrained XGBoost			Constrained RF XGBoost		
	Min	Max	Step	Min	Max	Step
<code>n_estimators</code>	5	100	1	1	30	1
<code>learning_rate</code>	0.1	0.5	0.05	0.1	0.5	0.05
<code>max_depth</code>	1	3	1	1	3	1
<code>num_parallel_tree</code>	1	1	1	2	3	1
<code>gamma</code>	0	1	0.1	0	1	0.1
<code>subsample</code>	0.5	1	0.05	0.5	1	0.05
<code>colsample_bytree</code>	0.5	1	0.01	0.5	1	0.01
<code>min_child_weight</code>	10	50	1	10	50	1
<code>alpha</code>	0	30	0.005	0	30	0.005
<code>lambda</code>	0	30	0.005	0	30	0.005
<code>grow_policy</code>	depthwise	lossguide	-	depthwise	lossguide	-

⁸Optuna: A hyperparameter optimization framework <https://optuna.org/>

Table 3. Fitted Hyperparameter Values

Hyperparameter	Constrained XGBoost	Constrained RF XGBoost
n_estimators	90	29
learning_rate	0.5	0.4
max_depth	1	2
num_parallel_tree	1	3
gamma	0.4	0.5
subsample	0.95	0.65
colsample_bytree	0.57	0.92
min_child_weight	21	20
alpha	2.93	0.39
lambda	14.15	23.085
grow_policy	depthwise	lossguide

4 PERFORMANCE

We measure two dimensions of model performance: discriminatory power and calibration accuracy. We follow applied conventions of using two metrics to assess rank-ordering power and evaluate the accuracy of probabilistic predictions. Empirical results were obtained from calculating metrics on a holdout set (30%).

4.1 Discriminatory power

Discriminatory power measures the rank-ordering ability of a model in order to quantify its ability at differentiating good and bad risk. In credit risk, the most commonly used metric for assessing discriminatory power is the Gini score sometimes also denoted in the risk literature as Accuracy Ratio [10], [23].

$$\text{Gini} = P[S_D < S_N] - P[S_D > S_N] \quad (7)$$

In Equation 7, the Gini score represents a probability that a randomly chosen example from a positive class (defaulted, S_D) is ranked higher than a randomly chosen example from the negative class (non-defaulted, S_N) by a scoring model.

The closer the Gini score is to one the better. AUC score can be derived from the Gini score and vice versa as follows: $\text{AUC} = (1 + \text{Gini}) / 2$, $\text{Gini} = 2 * \text{AUC} - 1$.

4.2 Calibration accuracy

A significant drawback of rank-ordering metrics like the Gini score is their inability to assess the actual accuracy of the probabilistic predictions. To address this concern, model calibration accuracy is assessed separately by categorizing predictions into bins (or risk grades) and applying various statistical tests.

A common metric used in measuring the calibration of probabilistic predictions without assigning a subjective number of bins is the Brier score [5]. Brier score is a proper scoring rule that equates to Mean Squared Error (MSE) for binary classification. Brier score can be decomposed into calibration and refinement loss [18], [2], [12], [1] shown in Equations 9 and 10.

$$\text{Brier score} = \frac{1}{n} \sum_{j=1}^n (y_j - p_j)^2 \quad (8)$$

$$\text{Calibration loss} = \frac{1}{n} \sum_{k=1}^n \sum_{j=n_i} (r_i - p_i)^2 \quad (9)$$

$$\text{Refinement loss} = \frac{1}{n} \sum_{k=1}^n \sum_{j=n_i} r_i(1 - r_i) \quad (10)$$

The Brier score in Equation 8 measures the mean squared error between actual outcomes (y_j) and predicted probabilities (p_j) for each data point. r_i in Equations 9 and 10 corresponds to average event (default) rate while p_i corresponds to average predicted probability (of default) for each unique ROC segment.

The partitioning of the Brier score allows us to establish a calibration loss metric, which calculates the mean squared deviation from empirical probabilities based on ROC segments (L1 norm). Alternatively, we can adjust the formula in Equation 9 to derive the mean absolute error, L2 norm. This metric definition is conceptually related to Expected Calibration Error (ECE) as in [14], but in this paper it is measured at the ROC segment level without discretizing predictions into bins. We then derive the calibration score as $1 - \text{ECE}$, offering a way to evaluate predictive performance similar to the Gini score's role in conveying model performance to business stakeholders. Visual representation still employs equally distributed bins in a calibration curve diagram.

4.3 Experimental results

The Cumulative Accuracy Profile (CAP) curve in the left hand-side plot of Figure 2 shows discriminatory power for the three scoring models. We observe that at 10% of our predictions sorted by highest risk in descending order (red line), both tree-based models achieve equal performance capturing around 70% of the bad risk. WOE LR's discriminatory power is more modest capturing only 50%. Hence, we can derive from this that at the 10% of highest risk, the gain in discriminatory power is equal to 20% in absolute terms compared to the overall absolute difference in Gini scores of 15%.

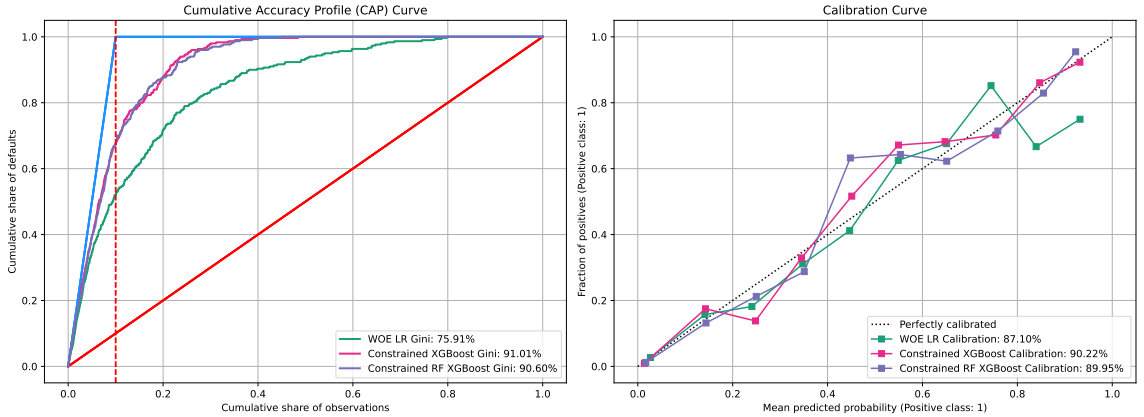


Fig. 2. Discriminatory power and calibration accuracy.

In the right hand-side plot of Figure 2, we report a calibration diagram for the three models using 10 equal bins for visualization. Based on the calibration scores, we can observe that Constrained XGBoost has a 3.12% better calibration than WOE LR. The direction of calibration scores is very similar to the discriminatory power ranking of models where Constrained XGBoost has the best discriminatory and predictive performance.

5 INTERPRETABILITY

Understanding the inner workings of models is a critical requirement in high-stakes decision-making, such as in the lending business. To interpret the models' outputs and compare the most important risk drivers within each model, we employ the SHAP (SHapley Additive exPlanations) framework. SHAP values represent feature "payouts" by quantifying each feature's average marginal contribution to predictions across different feature coalitions [17].

Below, SHAP feature importance plots are visualized in Figure 3 highlighting most important risk drivers for each model's outputs. Second, we provide plots of feature importances calculated using average contributions of each feature's score points to the final credit score in Figure 4. Finally, we show in Figure 5 distributions of scores between good and bad risk to illustrate similarity of model outputs for end users.

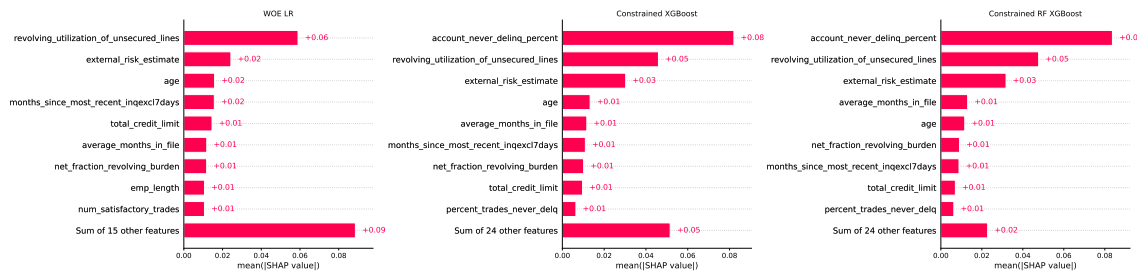


Fig. 3. SHAP importance plots.

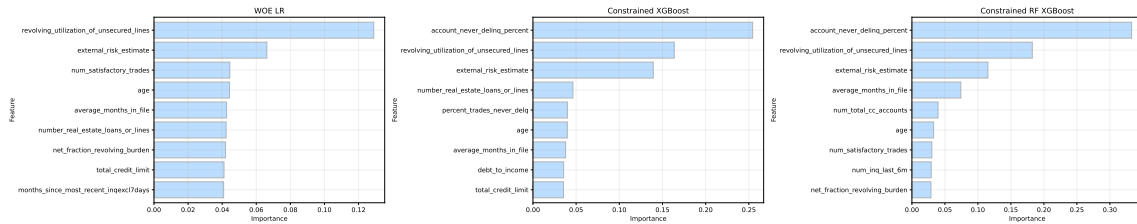


Fig. 4. Scorecard importance plots.

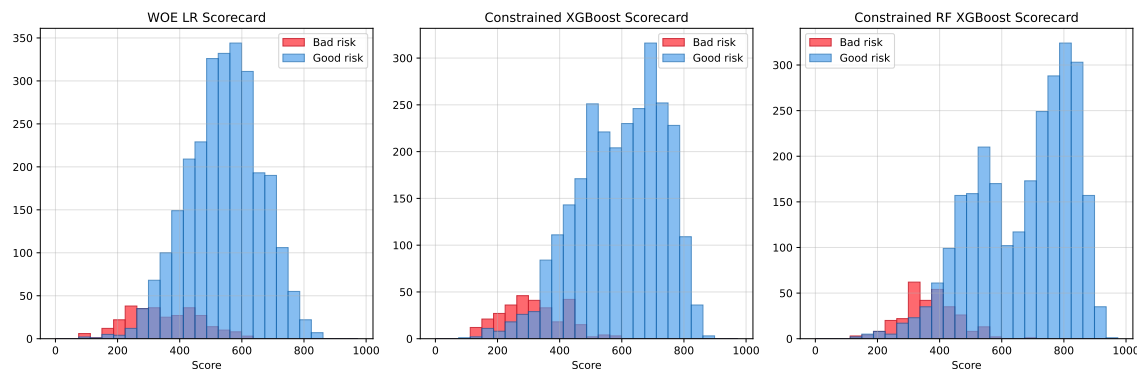


Fig. 5. Distribution of scores.

We can observe the overall consistency of results noting, however, that the feature importance plot for Constrained RF XGBoost scorecard highlighted one variable that SHAP did not include: `num_total_cc_accounts`. This feature describes a number of credit cards a customer has and has an increasing risk trend with regards to default risk. Feature `account_never_delinq_percent` was not included in the WOE LR model’s short-list due to a zero coefficient. Additionally, it can be seen from the scorecard plots in Figure 5 that there are two distributions of good risk in the Constrained RF XGBoost scorecard (more pronounced than in the Constrained XGBoost score distribution). The distributions shown in Figure 5 indicate possible alternative score cut-offs that a lender can employ to perform underwriting policy testing.

6 CONCLUSION

Drawing upon previous studies [9], [8], [15], this study demonstrates using a synthetic dataset created by blending real-world credit risk data that when responsibly designed and properly constrained, tree-based ML models for credit scoring – under certain assumptions – outperform their linear alternatives both in terms of discriminatory power and calibration accuracy while remaining highly interpretable. A method of transforming tree-based models into scorecards discussed in this paper may promote acceptance of such models in decision-making and make a compelling case for cautious lenders to begin using more advanced algorithms without having to compromise on the balance between explainability and accuracy.

This research underscores the potential of ML to further revolutionize consumer lending, offering a bridge between traditional risk assessment methods and cutting-edge predictive models. By drawing from the principles of responsible design and explainability, lenders can harness the full potential of ML models in credit risk management, contributing to more robust, efficient, and profitable lending practices.

7 ACKNOWLEDGMENTS

The author would like to thank Paul Edwards for providing helpful critique on the draft and sharing the approach, Alexey Calabourdin (a.calabourdin@gmail.com) for helpful tips on calibration and sharing the Brier score decomposition code, Fabiana Clemente and and Guillermo Navas-Palencia for providing inputs on the draft.

A APPENDIX

Table 4 reports an overview of the data sources used to create a synthetic dataset, including monotonic constraints (+1 indicates increasing risk with an increase in feature values and −1 indicates the opposite direction).

The most up-to-date code base for this paper is available in the following GitHub repo: <https://github.com/deburky/xml-scoring>.

Table 4. Dataset description

Dataset	External name	Attribute name	Standardized name	Description	Causal
Dataset A	FICO (HELOC)	RiskPerformance	is_bad	90+ DPD over 24 months	Target
Dataset A	FICO (HELOC)	ExternalRiskEstimate	external_risk_estimate	Consolidated risk markers	-1
Dataset A	FICO (HELOC)	MSinceOldestTradeOpen	months_since_oldest_trade_open	Months since oldest trade	-1
Dataset A	FICO (HELOC)	MSinceMostRecentTradeOpen	months_since_most_recent_trade_open	Months since most recent trade	-1
Dataset A	FICO (HELOC)	AverageMinFile	average_months_in_file	Average months in file	-1
Dataset A	FICO (HELOC)	NumSatisfactoryTrades	num_satisfactory_trades	Number satisfactory trades	-1
Dataset A	FICO (HELOC)	NumTrades60Ever2DerogPubRec	num_trades_60_ever_2_derog_pub_rec	Number trades 60+ Ever	1
Dataset A	FICO (HELOC)	NumTrades90Ever2DerogPubRec	num_trades_90_ever_2_derog_pub_rec	Number trades 90+ Ever	1
Dataset A	FICO (HELOC)	PercentTradesNeverDelq	percent_trades_never_delq	Percent trades never delinq	-1
Dataset A	FICO (HELOC)	MSinceMostRecentDelq	months_since_most_recent_delq	Months since most recent delinq	-1
Dataset A	FICO (HELOC)	NumTradesOpeninLast12M	num_trades_open_in_last_12m	Number of trades open in last 12 months	1
Dataset A	FICO (HELOC)	MSinceMostRecentInqexcl7days	months_since_most_recent_inqexcl7days	Months since most recent inquiry (excl 7 days)	-1
Dataset A	FICO (HELOC)	NumInqLast6M	num_inq_last_6m	Number of inquiries last 6 months	1
Dataset A	FICO (HELOC)	NumInqLast6Mexcl7days	num_inq_last_6m_excl7days	Number of inquiries last 6 months (excl 7 days)	1
Dataset A	FICO (HELOC)	NetFractionRevolvingBurden	net_fraction_revolving_burden	Net fraction revolving burden	1
Dataset A	FICO (HELOC)	NetFractionInstallBurden	net_fraction_install_burden	Net fraction installment burden	1
Dataset A	FICO (HELOC)	NumBank2NatlTradesWHighUtilization	num_bank_2_natl_trades_w_high_utilization	Number of trades with high utilization	1
Dataset B	Lending Club	emp_length	emp_length	Number of years in job (max 10)	-1
Dataset B	Lending Club	annual_income	annual_income	Annual income	1
Dataset B	Lending Club	debt_to_income	debt_to_income	Debt-to-income ratio	1
Dataset B	Lending Club	delinq_2y	is_bad	Delinquencies on lines of credit in last 2 years	Target
Dataset B	Lending Club	total_credit_limit	total_credit_limit	Total available credit (excl mortgage)	1
Dataset B	Lending Club	total_credit_utilized	total_credit_utilized	Total credit balance (excl mortgage)	1
Dataset B	Lending Club	num_historical_failed_to_pay	num_historical_failed_to_pay	Number of derogatory public records	1
Dataset B	Lending Club	current_installment_accounts	current_installment_accounts	Number of installment accounts	1
Dataset B	Lending Club	num_total_cc_accounts	num_total_cc_accounts	Total number of credit card accounts	1
Dataset B	Lending Club	num_open_cc_accounts	num_open_cc_accounts	Total number of open credit card accounts	1
Dataset B	Lending Club	num_cc_carrying_balance	num_cc_carrying_balance	Number of credit cards carrying balance	1
Dataset B	Lending Club	num_mort_accounts	num_mort_accounts	Number of mortgage accounts	-1
Dataset B	Lending Club	account_never_delinq_percent	account_never_delinq_percent	Percent of lines never delinquent	1
Dataset B	Lending Club	loan_amount	loan_amount	The amount of the loan received	1
Dataset B	Lending Club	balance	balance	Current balance on the loan	1
Dataset B	Lending Club	paid_total	paid_total	Total paid on the loan	-1
Dataset C	Give Me Some Credit	SeriousDlqin2yrs	is_bad	Person experienced 90+ days past due	Target
Dataset C	Give Me Some Credit	RevolvingUtilizationOfUnsecuredLines	revolving_utilization_of_unsecured_lines	Total balance on credit cards divided by credit limits	1
Dataset C	Give Me Some Credit	age	age	Age of borrower in years	-1
Dataset C	Give Me Some Credit	DebtRatio	debt_ratio	Monthly debt payments divided by monthly income	-1
Dataset C	Give Me Some Credit	MonthlyIncome	monthly_income	Monthly income	-1
Dataset C	Give Me Some Credit	NumberOfOpenCreditLinesAndLoans	number_of_open_credit_lines_and_loans	Number of open loans and lines of credit	1
Dataset C	Give Me Some Credit	NumberRealEstateLoansOrLines	number_real_estate_loans_or_lines	Number of mortgage and real estate loans	1
Dataset C	Give Me Some Credit	NumberOfDependents	number_of_dependents	Number of dependents in family	1

REFERENCES

- [1] Antonio Bella, Cèsar Ferri, José Hernández-Orallo, and María José Ramírez-Quintana. 2010. Calibration of Machine Learning Models. *IGI Global eBooks* (01 2010), 128–146. <https://doi.org/10.4018/978-1-60566-766-9.ch006>
- [2] Gail Blattenberger. 2023. Separating the Brier Score into Calibration and Refinement Components: A Graphical Exposition. <https://www.tandfonline.com/doi/abs/10.1080/00031305.1985.10479382>
- [3] Joseph Breeden. 2010. Reinventing Retail Lending Analytics. <https://riskbooks.com/reinventing-retail-lending-analytics-2nd-impression-1>
- [4] Joseph Breeden. 2020. Survey of Machine Learning in Credit Risk. *Journal of Credit Risk* 17 (09 2020), 1–62. <https://doi.org/10.13140/RG.2.2.14520.37121>
- [5] Glenn W. Brier. 1950. Verification of forecasts expressed in terms of probability. *Monthly Weather Review* 78 (01 1950), 1–3. [https://doi.org/10.1175/1520-0493\(1950\)078<0001:VOFEIT>2.0.CO;2](https://doi.org/10.1175/1520-0493(1950)078<0001:VOFEIT>2.0.CO;2)
- [6] Tianqi Chen and Carlos Guestrin. 2016. XGBoost: A Scalable Tree Boosting System. <https://dl.acm.org/doi/10.1145/2939672.2939785>
- [7] Yujia Chen, Raffaella Calabrese, and Belen Martín-Barragán. 2023. Interpretable machine learning for imbalanced credit scoring datasets. *European Journal of Operational Research* 312 (06 2023). <https://doi.org/10.1016/j.ejor.2023.06.036>
- [8] Paul Edwards. 2020. Machine Learning in Retail Credit Risk: Algorithms, Infrastructure, and Alternative Data — Past, Present, and Future. <https://www.nvidia.com/en-us/on-demand/session/gtcspring21-s31327/>
- [9] Paul K. Edwards, Dina Duhon, and Suhail Shergill. 2017. Real AdaBoost: boosting for credit scorecards and similarity to WOE logistic regression. <https://support.sas.com/resources/papers/proceedings17/1323-2017.pdf>
- [10] Bernd Engelmann, Evelyn Hayden, and Dirk Tasche. 2003. Measuring the Discriminative Power of Rating Systems. *Discussion Paper Series 2: Banking and Financial Studies* 01 (01 2003). <https://doi.org/10.2139/ssrn.2793951>
- [11] Experian. 2022. Explainability: ML and AI in Credit Decisioning. <https://experianacademy.com/blog/2022/04/11/new-insight-report-explainability-ml-and-ai-in-credit-decisioning/>
- [12] Peter Flach and Edson Takashi Matsubara. 2008. On classification, ranking, and Probability Estimation. *Dagstuhl Seminar Proceedings* 1 (01 2008), 1–10. <https://doi.org/10.4230/DagSemProc.07161.8>

- [13] Léo Grinsztajn, Edouard Oyallon, and Gaël Varoquaux. 2022. Why Do tree-based Models Still Outperform Deep Learning on Typical Tabular data? *Hal.science* (11 2022). <https://doi.org/hal-03723551>
- [14] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. 2017. On Calibration of Modern Neural Networks. <https://dl.acm.org/doi/10.5555/3305381.3305518>
- [15] Patrick Hall, James Curtis, and Parul Pandey. 2023. Machine Learning for High-Risk Applications: Approaches to Responsible AI. <https://www.oreilly.com/library/view/machine-learning-for/9781098102425/>
- [16] Olga Loiseau-Aslanidi, Natche Subramaniam Thiagarajah, and Vera Tolstova. 2020. Automating Interpretable Machine Learning Scorecards. <https://www.moodysanalytics.com/-/media/article/2020/Automating-Interpretable-Scorecards.pdf>
- [17] Christoph Molnar. 2023. Interpretable Machine Learning. <https://christophm.github.io/interpretable-ml-book/>
- [18] Allan H Murphy. 1972. Scalar and Vector Partitions of the Probability Score: Part I. Two-State Situation. *Journal of Applied Meteorology and Climatology* 11 (03 1972), 273–282. [https://doi.org/10.1175/1520-0450\(1972\)011<0273:SAVPOT>2.0.CO;2](https://doi.org/10.1175/1520-0450(1972)011<0273:SAVPOT>2.0.CO;2)
- [19] Guillermo Navas-Palencia. 2020. Optimal binning: mathematical programming formulation. <https://arxiv.org/abs/2001.08025>
- [20] Rob Potharst and Ad Feelders. 2002. Classification Trees for Problems with Monotonicity Constraints. *SIGKDD Explorations* 4 (06 2002), 1–10. <https://doi.org/10.1145/568574.568577>
- [21] Jessica Renier, Conan French, and Daniel Mendez Delgado. 2022. IIF and EY 2022 Survey Report on Machine Learning. <https://www.iif.com/Publications/ID/5184/IIF-and-EY-2022-Survey-Report-on-Machine-Learning--Summary>
- [22] Cynthia Rudin. 2019. Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nature Machine Intelligence* 1 (05 2019), 206–215. <https://doi.org/10.1038/s42256-019-0048-x>
- [23] Marco van der Burgt. 2005. Calibration and mapping of credit scores by riding the cumulative accuracy profile. <https://www.risk.net/journal-of-credit-risk/6405871/calibration-and-mapping-of-credit-scores-by-riding-the-cumulative-accuracy-profile>
- [24] Corey Wade and Kevin Glynn. 2020. *Hands-On Gradient Boosting with XGBoost and scikit-learn*. Packt Publishing Ltd. <https://www.packtpub.com/product/hands-on-gradient-boosting-with-xgboost-and-scikit-learn/9781839218354>
- [25] Chen Wang, Chengyuan Deng, and Shuzhen Wang. 2020. Imbalance-XGBoost: Leveraging Weighted and Focal Losses for Binary label-imbalanced Classification with XGBoost. *Pattern Recognition Letters* 136 (08 2020), 190–197. <https://doi.org/10.1016/j.patrec.2020.05.035>